Assignment 5: Optimising Attention on a CPU

CS6886 Systems for Deep Learning

November 2026

All numbers were measured on the machine described in Q1 with clang 23.1.4 and the Makefile's flags (`-O3 -march=native`, plus `-fopenmp` for `omp`). FLOP counts follow the driver: $4N^2d$ per call (two $N \times N \times d$ matrix products; softmax work is not counted). The machine is a shared login node, so I pinned single-threaded runs to one idle core (`taskset -c 10 make <variant>`). I ran every `make` target 3 times and report the best of the 3 driver medians. Each driver median is already the median of 5 timed runs after a warm-up. OpenMP runs used `OMP_PROC_BIND=close` and `OMP_PLACES={24}:24,{0}:24` (socket 1 first, which is the quieter socket) and report the best of 7 launches. `make check` passes for every variant and input (largest relative error 7.5×10^{-6}; the limit is 10^{-4}).

Q1. Machine

CPU	2 × Intel Xeon Gold 6240R, Cascade Lake-SP, 2 sockets
Cores	48 physical = 48 logical (no SMT); all identical (no P/E split)
Clocks	base 2.4 GHz, max turbo 4.0 GHz (spec)
SIMD	AVX-512 (16 FP32 lanes), 2 × 512-bit FMA units per core, 32 zmm registers
Caches	L1d 32 KB/core, L2 1 MB/core (private), L3 35.75 MB/socket (shared by 24 cores), 64 B lines
Memory	192 GB DDR4 (12 × 16 GB), 6 channels/socket, DDR4-2933
Software	AlmaLinux 8.9, Linux 4.18, clang 23.1.4

Sources: `lscpu`, `/sys/devices/system/cpu`, `/sys/devices/system/edac` (DIMM sizes and channels) and Intel's specification. `dmidecode` needs root, so I could not read the DIMM speed. I used 2933 MT/s, the highest speed the 6240R supports, and the measured bandwidth below is consistent with it.

Measured clocks and peaks. The spec turbo clock does not hold under AVX-512 load, so I measured the real numbers with small benchmarks (kept outside the repository):

- *Clock.* A chain of dependent 1-cycle integer adds runs at 3.59 GHz on one core and 3.18 GHz on all 48 cores.
- *FMA peak.* 16 independent chains of `vfmaddps zmm` per thread (the disassembly is 16 FMAs and a loop branch, with no spills) give 179.5 GFLOP/s on one core. That is 179.5/64 = 2.81 GHz, the AVX-512 clock with both FMA ports busy (AVX2: 101.6 GFLOP/s at 3.17 GHz). On all 48 cores it gives 4433 GFLOP/s, an effective 1.43 GHz. The CPU drops to its AVX-512 power licence and the 165 W socket limit. Per core: 179.5 (1 core), 157.7 (8), 112.7 (24), 92 (48).
- *Bandwidth.* A streaming read of 512 MB arrays gives 13.8 GB/s on one core and 232.2 GB/s on 48 cores (pages first-touched in parallel). For one core, the same test on smaller working sets gives 342 GB/s (L1), 133 GB/s (L2) and 24.8 GB/s (L3).

Derived numbers.

$$
\begin{aligned}
\text{FLOP/cycle/core} &= 2 \text{ FMA units} \times 16 \text{ lanes} \times 2 \text{ FLOP} = 64 \\
\text{Peak, one core} &= 64 \times 4.0 \text{ GHz} = 256 \text{ GFLOP/s (theoretical)} \\
&= 64 \times 2.81 \text{ GHz} = 179.5 \text{ GFLOP/s (measured, AVX-512 clock)} \\
\text{Peak, all cores} &= 48 \times 64 \times 2.4 \text{ GHz} = 7372.8 \text{ GFLOP/s (theoretical, base clock)} \\
&= 4433 \text{ GFLOP/s (measured, 1.43 GHz effective)} \\
\text{DRAM bandwidth} &= 2 \text{ sockets} \times 6 \text{ channels} \times 2933 \text{ MT/s} \times 8 \text{ B} = 281.6 \text{ GB/s (theoretical)} \\
&= 232.2 \text{ GB/s (measured, 82\%)}; \quad 13.8 \text{ GB/s from one core} \\
\text{Ridge, one core} &= 179.5/13.8 = 13.0 \text{ FLOP/B} \\
\text{Ridge, all cores} &= 4433/232.2 = 19.1 \text{ FLOP/B} \quad (7372.8/281.6 = 26.2 \text{ theoretical})
\end{aligned}
$$

One core cannot use the whole memory system: it has only about 12 outstanding L1 misses, so it reaches 13.8 GB/s. Its roof therefore uses the single-core bandwidth. I use the *measured* numbers for all percentages and for the roofline, because they are what code can actually reach on this machine.

Q2. Naive baseline

Table 1: `make naive`, one thread. Percent of the measured single-core peak (179.5 GFLOP/s).

N	d	time (ms)	GFLOP/s	% of peak
1024	64	124.3	2.16	1.20%
2048	64	500.6	2.14	1.19%
4096	64	2,258.1	1.90	1.06%
4096	128	8,133.7	1.06	0.59%

Where the time goes. I timed the three phases separately with identical code and flags:

input	$QK^\top$	softmax	SV	total
$N = 1024, d = 64$	46 ms (37%, 2.91 GFLOP/s)	5 ms (4%)	73 ms (59%, 1.83 GFLOP/s)	125 ms
$N = 2048, d = 64$	184 ms (37%, 2.91)	22 ms (4%)	296 ms (59%, 1.81)	502 ms
$N = 4096, d = 64$	738 ms (33%, 2.91)	88 ms (4%)	1441 ms (64%, 1.49)	2266 ms
$N = 4096, d = 128$	1877 ms (22%, 2.29)	87 ms (1%)	6420 ms (77%, 0.67)	8384 ms

- $QK^\top$ **is latency-bound.** The inner loop is a dot product `acc += Q[i][k]*K[j][k]`. Without `-ffast-math`, clang may not reorder FP additions, so it refuses to vectorise the reduction (`-Rpass-missed`: "cannot prove it is safe to reorder floating-point operations"). The code is one scalar `vfmadd231ss` chain, unrolled 8 times but serially dependent. At 4 cycles of FMA latency that is 2 FLOP per 4 cycles, about 1.8 GFLOP/s at 3.6 GHz. The measured 2.9 GFLOP/s is higher because the next j's chain is independent, and the out-of-order core overlaps the tail of one chain with the head of the next. With $d = 128$ the chains are twice as long, so less of them fits in the reorder window and the rate drops to 2.3 GFLOP/s.
- SV **is latency- and cache-bound.** The inner loop runs over j with `V[j*d+k]`, a column walk with a stride of $4d$ bytes. Each FMA touches a new cache line and is again one serial scalar chain. With $d = 128$ the stride is 512 B, so the column walk only uses 1/8 of the L2 sets (an effective 128 KB of L2). The 4096 lines touched between reuses of a line do not fit, so V comes from L3 and the rate falls to 0.67 GFLOP/s. With $d = 64$ (stride 256 B, 1/4 of the sets) it only just fits.
- **Softmax** is a small share (N^2 calls of scalar `expf`, about 5 ns each). It matters later, once the matrix products are fast.

Scaling with N and d. Work grows as $N^2 d$. From $N = 1024$ to 2048 the time grows 4.0×, exactly quadratic, because nothing changes cache level. From 2048 to 4096 it grows 4.5× (4096 × 4096 floats of S is 64 MB, more than the 35.75 MB L3), and the share of SV grows. Doubling d at $N = 4096$ costs 3.6×, not 2×. The $QK^\top$ chains overlap less, and the strided SV walk falls out of L2, as explained above. The naive code is at 0.6–1.2% of the single-core peak, so it is limited by FMA latency and cache misses, not by FLOPs or DRAM bandwidth.

Q3. Optimisations (single thread)

Each table lists all variants for one input; the plots `plots/time_N*_d*.png` show the same data.

Table 2: $N = 1024$, $d = 64$ (0.27 GFLOP per call). Peak: 179.5 GFLOP/s (one core), 4433 GFLOP/s (48 cores).

variant	threads	time (ms)	GFLOP/s	% of peak	speedup vs naive
naive	1	124.3	2.16	1.2	–
loops	1	21.39	12.55	7.0	5.8×
tiled	1	9.58	28.01	15.6	13.0×
fused	1	21.12	12.71	7.1	5.9×
best	1	2.06	130.50	72.7	60.4×
omp	24	0.207	1296.8	29.3*	600×

*percent of the 48-core peak.

Table 3: $N = 2048$, $d = 64$ (1.07 GFLOP per call). Peak: 179.5 GFLOP/s (one core), 4433 GFLOP/s (48 cores).

variant	threads	time (ms)	GFLOP/s	% of peak	speedup vs naive
naive	1	500.6	2.14	1.2	–
loops	1	85.42	12.57	7.0	5.9×
tiled	1	38.27	28.05	15.6	13.1×
fused	1	94.45	11.37	6.3	5.3×
best	1	7.71	139.27	77.6	64.9×
omp	48	0.555	1934.7	43.6*	902×

*percent of the 48-core peak.

Table 4: $N = 4096$, $d = 64$ (4.29 GFLOP per call). Peak: 179.5 GFLOP/s (one core), 4433 GFLOP/s (48 cores).

variant	threads	time (ms)	GFLOP/s	% of peak	speedup vs naive
naive	1	2,258.1	1.90	1.1	–
loops	1	380.8	11.28	6.3	5.9×
tiled	1	160.4	26.77	14.9	14.1×
fused	1	453.0	9.48	5.3	5.0×
best	1	30.70	139.91	77.9	73.6×
omp	48	1.673	2567.2	57.9*	1350×

*percent of the 48-core peak.

Table 5: $N = 4096$, $d = 128$ (8.59 GFLOP per call). Peak: 179.5 GFLOP/s (one core), 4433 GFLOP/s (48 cores).

variant	threads	time (ms)	GFLOP/s	% of peak	speedup vs naive
naive	1	8,133.7	1.06	0.6	–
loops	1	794.4	10.81	6.0	10.2×
tiled	1	189.3	45.38	25.3	43.0×
fused	1	776.4	11.06	6.2	10.5×
best	1	59.45	144.49	80.5	136.8×
omp	40	3.469	2476.2	55.9*	2345×

*percent of the 48-core peak.

(a) `attn_loops.c`: loop reordering

I transpose K once ($O(Nd)$), compute S in i-k-j order (`S[i][j] += Q[i][k]*Kt[k][j]`) and O in i-j-k order (`O[i][k] += S[i][j]*V[j][k]`). Every inner loop is now unit-stride with no reduction, so clang vectorises it under strict IEEE rules (8-wide AVX2; clang prefers 256-bit vectors on Skylake-family CPUs). Each output

still adds its terms in the same order as naive. Max, sum and normalise are separate loops; `expf` stays scalar. The speedup is 5.8–5.9× for $d = 64$ and 10.2× for $d = 128$, where naive suffered most from the strided walk.

What limits it now (measured per phase at $N = 4096, d = 64$: $QK^\top$ 125 ms, softmax 110 ms, SV 155 ms):

- *Per-row streaming of Kt and V.* For every row i the whole of Kt (Nd floats) is read for $2Nd$ FLOP, which is 0.5 FLOP/B from wherever Kt lives. At $d = 128$, Kt is 2 MB and lives in L3: 0.5×24.8 GB/s $= 12.4$ GFLOP/s, exactly the measured $QK^\top$ rate (12.4 GFLOP/s). At $d = 64$, Kt is 1 MB, partly in L2, and the rate is 17.1 GFLOP/s.
- *Accumulators in memory.* Each vector FMA needs two loads and a store, because the running sums (the S row, the O row) live in L1, not in registers. Even at $N = 1024$ (everything in L2) the products run at only 15–21 GFLOP/s.
- *Scalar `expf`.* It is now 28% of the time.

(b) `attn_tiled.c`: cache and register tiling

Both products use one AVX-512 micro-kernel $C_{6\times64}$ (+)= $A_{6\times k}B_{k\times64}$. Its 24 accumulators (6 rows × 4 zmm) stay in registers for the whole k loop. Each step does 4 vector loads of B, 6 broadcasts of A and 24 FMAs, so each B value is reused 6 times and each A value 64 times. I checked the inner loop in the disassembly: 24 `vfmadd231ps` and no spills.

- $QK^\top$: K is packed once into 64-key panels `Kp[p][k][0..63]`, so B rows are contiguous. Loops: 192-row block of Q (L2) → key panel (16–32 KB, L1/L2) → 6-row strip.
- SV: rows of S are A and V is B. Loops: 192-row block (its O block stays in L2) → 64-key block of V (L1) → strip → 64-column chunk. Edge tiles go through zero-padded copies, so any N, d works.

The two products now take 25 ms each at $N = 4096, d = 64$ (86 GFLOP/s) and 40 ms each at $d = 128$ (108 GFLOP/s). At $N = 4096$, this includes writing the 64 MB S to DRAM and reading it back. The 4 passes over S need at least 18.5 ms at 13.8 GB/s, plus kernel page zeroing of the fresh 64 MB buffer.

What limits it now: Amdahl. Softmax with scalar `expf` is unchanged at about 110 ms, so it is **68%** of the 160 ms at $N = 4096, d = 64$ (and 74% at $N = 1024$). Tiling sped up the products about 6× over `loops`, but the total only 2.4×. A vectorised exponential, and not storing S, are the next steps.

(c) `attn_fused.c`: row-wise fusion

For each query row the three steps run back to back on one row buffer (N floats, 16 KB at $N = 4096$, so it stays in L1). S is never stored, and normalisation is applied to the d outputs instead of the N probabilities. The inner loops are the same compiler-vectorised loops as in `loops`, so comparing the two isolates the effect of fusion. Result: equal at $N = 1024$, slightly *slower* at larger N (453 vs 381 ms at $N = 4096, d = 64$).

Why fusion alone does not help. It removes the $16N^2$ bytes of S traffic. That traffic was never the bottleneck of `loops`, though (64 MB × 4 at 13.8 GB/s ≈ 19 ms out of 381 ms). Fusion also makes each row need *both* Kt and V back to back. Their combined size ($2Nd$ floats: 2 MB at $N = 4096, d = 64$, 1 MB at $N = 2048$) exceeds the 1 MB L2. In `loops`, each phase only cycles through one of them. So in `fused` every row streams 2 MB from L3 at 0.5 FLOP/B. Its matrix part (453 ms minus 110 ms of softmax) runs at 12.5 GFLOP/s, exactly the L3 roof of 0.5×24.8 GB/s. At $N = 1024$ both fit in L2, and `fused` and `loops` tie. Fusion only pays off once rows are also *tiled*, so that each K/V element brought from L3 is reused by many query rows. That is what (d) does.

(d) `attn_best.c`: FlashAttention-style kernel

1. **Fusion with tiling and online softmax.** For a block of $B_r = 96$ query rows, the kernel loops over 64-key panels. For each 6-row strip it computes $S_{6\times64}$ in 24 zmm registers with the (b) micro-kernel, then does the softmax in registers. It keeps a running max m and sum ℓ per row: $m' = \max(m, \text{rowmax}\, S)$, $\alpha = 2^{m-m'}$, $P = 2^{cS-m'}$, $\ell \leftarrow \alpha\ell + \text{rowsum}\, P$. Then it updates $O_{6\times64} \leftarrow \alpha O + PV$ with the same micro-kernel. S and P never leave registers and a 1.5 KB L1 buffer. Each K/V panel brought into cache is reused by 96 query rows, which raises the L3 intensity from 0.5 to $B_r/2 = 48$ FLOP/B, above the ridge.
2. **Vectorised `exp2`.** Q's scale is $c = \log_2 e/\sqrt{d}$, applied inside the FMA $cS - m'$, so softmax needs 2^x. I compute it 16 lanes at a time: $n = \text{round}(x)$ (`vrndscaleps`), $f = x - n \in [-\frac{1}{2}, \frac{1}{2}]$, then a degree-5 minimax polynomial for 2^f, fitted by me with relative error 2.3×10^{-7} in FP32. Finally `vscalefps`$(p, n) = p \cdot 2^n$, which also flushes very negative inputs to 0. That is 8 vector instructions per 16 exponentials, against about 20 cycles per scalar `expf`.

3. **Vector-only reductions.** The row max uses a 4-step permute/max butterfly that leaves the max in every lane, so there is no vector-to-scalar round trip. The phases are ordered (all 6 maxima, then all α, then all 24 exponentials), so the six independent rows overlap in the out-of-order window. ℓ is kept as a 16-lane partial sum and reduced once at the end.
4. **Aligned packing.** K is packed into transposed 64-key panels. V is copied to a 64-byte-aligned buffer, because the driver's `malloc` gives 16-byte alignment and unaligned 64-byte loads split across cache lines.

Measured effect of each step ($N = 4096, d = 64$; each pair measured back to back, because machine speed drifts by about 10% over time on this shared node):

- First version (steps 1 and 4, $B_r = 48$, degree-6 polynomial, scalar max): 108 GFLOP/s.
- Block size $B_r = 6/12/24/48/96/192$ gives 65/85/99/109/121/122 GFLOP/s. Small blocks re-read K/V from L3 too often. $N = 1024$ is flat at about 104 GFLOP/s, because its K/V fit in L2.
- Replacing `exp2` by a no-op (an experiment only; the output is wrong) raises the rate from 118 to 147 GFLOP/s. So exponentials cost about 20% of the time, much more than their instruction count suggests: the softmax section is a latency-bound dependency chain.
- Step 3 with the degree-5 polynomial and the scale folded into the FMA: 118 → 133 GFLOP/s (+13%).
- Aligned V copy versus unaligned in-place V: +4% ($d = 64$), +7% ($d = 128$).

Result: 130–145 GFLOP/s, 73–80% of the measured single-core peak. Speedups are 60, 65, 74 and 137× over naive. **What limits it now:** the core's two 512-bit vector ports. Besides the FMAs, they also execute the exponentials, maxima, rescaling of O and the butterflies (about 15% extra vector instructions per tile). The softmax section is also partly latency-bound. Memory is not a limit: compulsory DRAM traffic is $16Nd$ bytes, and L3 traffic is 1/48 of the FLOPs.

Q4. OpenMP (`attn_omp.c`)

The kernel is `attn_best.c`. Query blocks are independent (each owns its rows of O and its m, ℓ), so they are shared out with `schedule(dynamic,1)` and need no synchronisation. The block size adapts to p threads: about 2 blocks per thread, a multiple of 6 rows, at most 96. K and V packing is also parallel, and each thread has private accumulators.

Table 6: `attn_omp`: best of 7 launches (each the median of 5 timed runs). Speedup $S = T_1/T_p$, efficiency $E = S/p$.

	$N = 1024, d = 64$			$N = 2048, d = 64$			$N = 4096, d = 64$			$N = 4096, d = 128$		
p	ms	S	E	ms	S	E	ms	S	E	ms	S	E
1	2.136	1.0	1.00	8.098	1.0	1.00	30.85	1.0	1.00	58.67	1.0	1.00
2	1.159	1.8	0.92	4.146	2.0	0.98	16.10	1.9	0.96	30.37	1.9	0.97
4	0.637	3.4	0.84	2.375	3.4	0.85	8.419	3.7	0.92	16.04	3.7	0.91
8	0.365	5.9	0.73	1.338	6.1	0.76	5.106	6.0	0.76	9.641	6.1	0.76
12	0.302	7.1	0.59	0.920	8.8	0.73	3.569	8.6	0.72	6.867	8.5	0.71
16	0.244	8.8	0.55	0.726	11.2	0.70	2.746	11.2	0.70	5.229	11.2	0.70
24	0.207	10.3	0.43	0.591	13.7	0.57	2.157	14.3	0.60	4.396	13.3	0.56
32	0.277	7.7	0.24	0.638	12.7	0.40	2.049	15.1	0.47	3.862	15.2	0.47
40	0.283	7.5	0.19	0.585	13.8	0.35	1.707	18.1	0.45	3.469	16.9	0.42
48	0.295	7.2	0.15	0.555	14.6	0.30	1.673	18.4	0.38	23.52	2.5	0.05

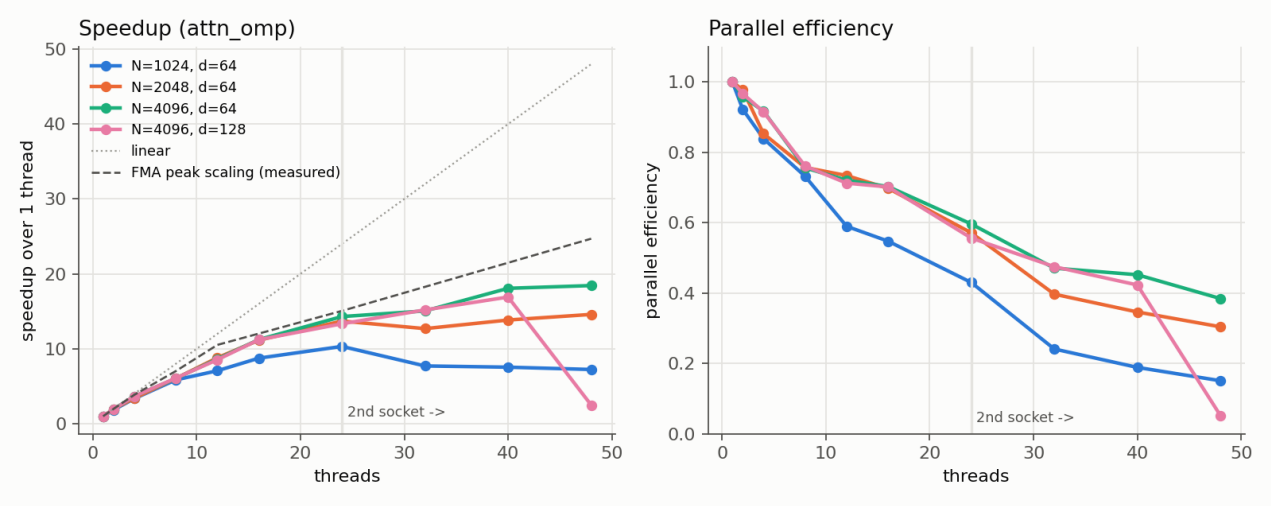


Figure 1: Speedup and efficiency of `attn_omp`. The dashed line is the speedup the pure FMA microbenchmark gets from p cores, which is the realistic ceiling for compute-bound code.

Explaining the scaling.

- **Efficiency is capped by the clock, not by the code.** With more active cores the CPU lowers its AVX-512 clock. The FMA benchmark itself only gets a 15.1× speedup from 24 cores and 24.7× from 48. At $N = 4096, d = 64$, `attn_omp` reaches 14.3× at 24 threads, 95% of that ceiling, and 18.4× at 48 threads (75%). Its best rate is 2567 GFLOP/s, 58% of the 48-core peak. The kernel stays compute-bound: per thread it needs only $B_r/2$ FLOP per byte of L3 traffic, and K/V (1–4 MB) fit in every socket's L3.
- **Small problems run out of work.** $N = 1024$ is 0.27 GFLOP, only 0.2 ms at 24 threads. With 171 six-row strips over 24 threads, the per-thread share is small. Then fork/join, packing (its own parallel region and barrier) and the uneven last blocks are a large fraction. Above 24 threads the second socket adds cross-socket traffic (Q, K and V were first-touched on one node) and slower barriers, so $N = 1024$ gets slower beyond 24 threads, and $N = 2048$ stops improving.
- **48 threads on a shared node.** At 48 threads every core is in use, including two cores where another user ran 100%-CPU processes, plus other periodic jobs. The Linux scheduler then time-slices one of our threads away for several milliseconds. The parallel region's barrier waits for it, so the whole call is late. I confirmed this by timing each block. In slow runs, every block took 0.7–1.6 ms as usual, but one thread reached the barrier about 10 ms after the others. This is why the medians at 48 threads are poor (for example $N = 4096, d = 64$: best 1.67 ms, median 7.6 ms), and why $N = 4096, d = 128$ never had a clean run at 48 threads (best 23.5 ms). At 40 threads, where all 48 cores are not needed, best and median agree within 6%. On a dedicated node the 48-thread points would follow the dashed ceiling.

Q5. Roofline

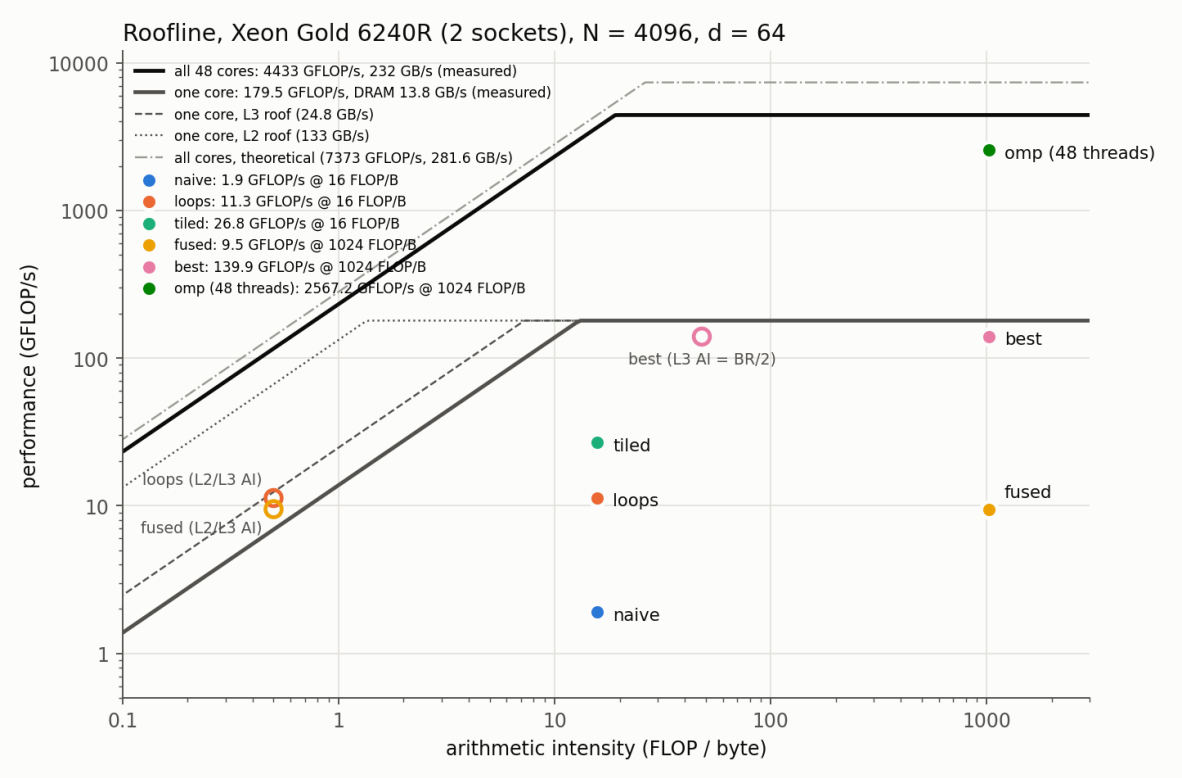


Figure 2: Roofline at $N = 4096$, $d = 64$ from the measured Q1 numbers. Filled markers use the DRAM arithmetic intensity. Hollow markers show the same runs at the intensity seen by the cache level that limits them.

How each intensity was computed. The FLOP count is always $4N^2d = 4.29$ GFLOP. Bytes are the DRAM traffic implied by each variant's data flow, with FP32 (4 B) data. Small re-reads that hit in cache are not counted.

- **naive, loops, tiled.** They read Q, K, V and write O ($4Nd$ floats), and S makes 4 trips through DRAM: written by $QK^\top$, read and written by softmax, read by SV. Within a row, softmax's three passes hit in L1/L2 (a row is 16 KB). So AI $= \dfrac{4N^2d}{16Nd + 16N^2} = \dfrac{Nd}{4(N+d)} = 15.8$ FLOP/B (31 at $d = 128$). The value is about $d/4$, so it is nearly independent of N.
- **fused, best, omp.** S never leaves the core, so only Q, K, V, O move: AI $= \dfrac{4N^2d}{16Nd} = \dfrac{N}{4} = 1024$ FLOP/B. The packed K and V copies stay in L3.
- **Cache-level intensities** (hollow markers). `loops` and `fused` re-read Kt and V from L2/L3 for every query row: $2Nd$ FLOP per $4Nd$ bytes $= 0.5$ FLOP/B. `best` re-reads them once per B_r-row block: $B_r/2 = 48$ FLOP/B.

What limits each variant (single-core roof for single-threaded runs).

- **naive** (1.9 GFLOP/s at 16 FLOP/B) sits far below both roofs, at 1% of peak. It is limited by FMA latency (scalar dependent chains) and by cache misses of the strided V walk, not by DRAM or FLOPs.
- **loops** (11.3) and **fused** (9.5) are bounded by the *L3 roof* at their cache-level intensity of 0.5 FLOP/B: $0.5 \times 24.8 = 12.4$ GFLOP/s, plus the scalar `expf` time. Their DRAM points look far from any roof because their real bottleneck is one cache level up. Fusion moved `fused` to the right in DRAM terms (16 → 1024 FLOP/B) without moving it up, since the L3 traffic stayed the same.
- **tiled** (26.8) has the same DRAM intensity as `loops`, above the single-core ridge of 13.0, so in principle it is compute-bound. Its matrix products do run at 86–108 GFLOP/s. The time is dominated by scalar `expf` (68%), which executes no counted FLOPs, so the point sits low: instruction-bound in the softmax.

- **best** (139.9, 78% of peak) is compute-bound on both axes: 1024 FLOP/B in DRAM terms and 48 FLOP/B in L3 terms, both right of the ridge. The gap to the roof is the non-FMA vector work of softmax on the same two ports.
- **omp** (2567 GFLOP/s at 48 threads, 58% of the 48-core peak) has the same intensity, far right of the 19.1 FLOP/B ridge, so it is compute-bound. The gap is the AVX-512 clock drop that the measured peak already includes, imperfect scaling across sockets, and the OS interference described in Q4.